CENTRO UNIVERSITÁRIO DE BRASÍLIA — CEUB
Educação a Distância
ADS

Ian Pedro Passos de Santana

# PLANEJAMENTO LÓGICO DO TASKTRACKER: SISTEMA DE GERENCIAMENTO DE TAREFAS

Desafio cumulativo — Fase 1

Brasília
2026
Ian Pedro Passos de Sanatana

# PLANEJAMENTO LÓGICO DO TASKTRACKER: SISTEMA DE GERENCIAMENTO DE TAREFAS

Desafio cumulativo — Fase 1

Documento de planejamento lógico apresentado ao Centro Universitário de Brasília — CEUB como requisito parcial para avaliação da disciplina Bootcamp II, na modalidade a distância, referente à Entrega Inicial (Fase 1) do desafio cumulativo.

Curso: ADS
Polo/Turma: Asa norte/ TURMA A - 0726]
E-mail institucional: ian.santana@sempreceub.com

Brasília
2026

## RESUMO

Este documento apresenta o planejamento lógico do TaskTracker, um sistema de gerenciamento de tarefas desenvolvido como desafio cumulativo da disciplina Bootcamp II. O trabalho corresponde à Fase 1 do projeto e tem por objetivo definir, antes da codificação, o problema a ser resolvido, os dados manipulados pelo

sistema, as regras que ele deve respeitar, o algoritmo que descreve o seu funcionamento e a estrutura do repositório que será publicado no GitHub. O problema investigado é a desorganização das tarefas cotidianas de quem concilia estudo e trabalho, situação em que a ausência de um registro estruturado provoca esquecimento de compromissos, perda de prazos, retrabalho e falta de visão do conjunto. Como solução, propõe-se um sistema executado em terminal que permite cadastrar tarefas com título, descrição, prioridade e data limite, listá-las ordenadas por importância e prazo, marcar as concluídas, removê-las e consultar um resumo geral. A decomposição do problema identificou oito entradas de dados, nove saídas e doze regras de sistema, todas com as validações correspondentes. O algoritmo foi descrito em português, passo a passo, e também representado em pseudocódigo, de modo a permitir a tradução direta para a linguagem Python na Fase 2. O mapeamento arquitetural definiu a árvore de pastas do repositório, o modelo conceitual das quatro áreas do Git e a sequência de comandos que será utilizada no versionamento. Conclui-se que o planejamento prévio reduz a ambiguidade da implementação e converte a etapa seguinte do projeto em uma tradução do que já foi decidido, e não em uma nova tomada de decisão.

**Palavras-chave:** Algoritmos. Lógica de programação. Gerenciamento de tarefas. Controle de versão. Git.

## LISTA DE ILUSTRAÇÕES

Figura 1 — Estrutura do repositório tasktracker-projeto ..... 17
Figura 2 — As quatro áreas do Git e o caminho de uma alteração ..... 20
Figura 3 — Organização do histórico com branches e merge ..... 21

## LISTA DE QUADROS

Quadro 1 — Funcionalidades do TaskTracker por fase do projeto ..... 8
Quadro 2 — Entradas de dados do sistema ..... 9
Quadro 3 — Saídas de dados do sistema ..... 9
Quadro 4 — Pseudocódigo do fluxo principal do TaskTracker ..... 14
Quadro 5 — Estrutura de pastas e arquivos do repositório ..... 17
Quadro 6 — Finalidade de cada item da estrutura do repositório ..... 17
Quadro 7 — Evolução prevista da estrutura nas Fases 2 e 3 ..... 18
Quadro 8 — As quatro áreas do Git ..... 19
Quadro 9 — Conceitos centrais do modelo do Git ..... 20
Quadro 10 — Aplicação do modelo do Git ao TaskTracker na Fase 2 ..... 21
Quadro 11 — Objetivos específicos de aprendizado de Git e GitHub ..... 22
Quadro 12 — Conformidade com os requisitos técnicos da gravação ..... 23

## SUMÁRIO

**1 APRESENTAÇÃO E ENTENDIMENTO DO PROBLEMA ..... 6** 1.1 Identificação ..... 6 1.2 O problema a ser resolvido ..... 6 1.3 A solução proposta: TaskTracker ..... 7
**2 DECOMPOSIÇÃO DO PROBLEMA E DADOS DO SISTEMA ..... 9** 2.1 Entradas de dados ..... 9 2.2 Saídas de dados ..... 9 2.3 Regras do sistema ..... 10
**3 RECEITA LÓGICA DO SISTEMA (ALGORITMO EM PORTUGUÊS) ..... 12** 3.1 Fluxo principal (menu do sistema) ..... 12 3.2 Passo a passo do cadastro de tarefa ..... 12 3.3 Passo a passo da visualização das tarefas ..... 13 3.4 Passo a passo da conclusão de uma tarefa ..... 13 3.5 Passo a passo da remoção de uma tarefa ..... 14 3.6 Passo a passo do resumo geral ..... 14 3.7 Representação em pseudocódigo ..... 14
**4 MAPEAMENTO ARQUITETURAL PARA GIT E GITHUB ..... 17** 4.1 Desenho da árvore do repositório ..... 17 4.1.1 Finalidade de cada item da estrutura ..... 17 4.1.2 Evolução prevista da estrutura (Fases 2 e 3) ..... 18 4.2 Modelo conceitual do Git ..... 18 4.2.1 As quatro áreas do Git ..... 19 4.2.2 O caminho de uma alteração até o GitHub ..... 19 4.2.3 Conceitos centrais do modelo ..... 20 4.2.4 Como o histórico se organiza com branches ..... 21 4.2.5 Como este modelo será aplicado ao TaskTracker na Fase 2 ..... 21 4.3 Plano de aprendizado para Git e GitHub ..... 22 4.3.1 Objetivos específicos de aprendizado ..... 22
**5 LINK DA APRESENTAÇÃO EM VÍDEO ..... 23** 5.1 Links da entrega ..... 23 5.2 Conformidade com os requisitos técnicos da gravação ..... 23 5.3 Checklist de entrega ..... 23
**6 CONSIDERAÇÕES FINAIS ..... 25**
**REFERÊNCIAS ..... 26**



## 1 APRESENTAÇÃO E ENTENDIMENTO DO PROBLEMA

### 1.1 Identificação

| Campo | Preenchimento |
|---|---|
| Nome completo | Ian Pedro Passos de Sanatana |
| Curso | ADS |
| Polo / Turma | Asa norte/ TURMA A - 0726 |
| E-mail institucional | ian.santana@sempreceub.com |
| Disciplina | Bootcamp II |
| Atividade | Desafio — Entrega Inicial (Fase 1) |

### 1.2 O problema a ser resolvido

A rotina de quem estuda e trabalha ao mesmo tempo é formada por uma quantidade grande de compromissos pequenos: uma atividade da faculdade que vence no fim da semana, uma tarefa do trabalho que depende de outra pessoa, uma conta para pagar, um capítulo de apostila para revisar. Quando não existe um lugar único e organizado para registrar tudo isso, o controle passa a ser feito pela memória ou por anotações espalhadas — um bloco de papel na mesa, uma mensagem enviada para si mesmo no WhatsApp, um arquivo de texto solto na área de trabalho e vários lembretes perdidos no celular.

O resultado desse cenário é sempre o mesmo: informação fragmentada e nenhuma visão do conjunto. A pessoa não consegue responder com segurança a três perguntas básicas — o que eu preciso fazer, o que é mais urgente agora e o que eu já concluí. Sem essas respostas, tarefas importantes são esquecidas até o prazo estourar, tarefas pouco relevantes consomem o tempo que deveria ser dedicado ao que é crítico e o trabalho já concluído acaba sendo refeito porque ninguém registrou que ele terminou. Em equipe o problema se multiplica: duas pessoas podem executar a mesma tarefa sem saber, e uma terceira pode ficar esperando por uma etapa que já estava pronta.

O problema, portanto, não é falta de esforço de quem organiza as tarefas: é a falta de um registro estruturado. As dores concretas que este projeto pretende atacar são:

- **Esquecimento —** tarefas que existem apenas na memória desaparecem quando a rotina fica cheia.

– **Ausência de priorização —** sem um critério explícito de importância, tudo parece igualmente urgente e a decisão de “o que fazer agora” é tomada no impulso.
– **Perda de prazos —** as datas limite ficam distribuídas em lugares diferentes e não são revisadas de forma sistemática.
– **Informação fragmentada —** papel, celular, e-mail e mensagens concorrem entre si e nenhum deles é a fonte oficial da informação.
– **Falta de histórico —** não há registro do que já foi concluído, o que gera retrabalho e impede qualquer avaliação do próprio progresso.
– **Ausência de visão do todo —** é impossível dizer, em um único olhar, quantas tarefas estão pendentes e quantas foram finalizadas.

### 1.3 A solução proposta: TaskTracker

O TaskTracker é um sistema de gerenciamento de tarefas que centraliza, em um só lugar, tudo o que o usuário precisa fazer. A proposta é deliberadamente simples e direta: em vez de oferecer dezenas de recursos, o sistema resolve bem o essencial — registrar uma tarefa com prioridade e prazo, mostrar a lista organizada por importância e permitir marcar o que já foi concluído. Assim o usuário passa a ter uma fonte única de informação e consegue responder, a qualquer momento, às três perguntas que hoje ficam sem resposta.

Na Fase 2 o sistema será implementado em Python e operado pelo terminal, por meio de um menu de opções numeradas. O usuário escolhe a operação desejada, o sistema pede as informações necessárias, valida o que foi digitado e devolve uma resposta clara na tela. O funcionamento previsto é o seguinte:

– **Cadastrar tarefa —** o usuário informa título, descrição (opcional), prioridade (Alta, Média ou Baixa) e data limite; o sistema valida os dados, gera um identificador único e grava a tarefa com o status “Pendente”.
– **Visualizar tarefas —** o sistema exibe a lista completa em formato de tabela, ordenada da prioridade mais alta para a mais baixa e, dentro da mesma prioridade, do prazo mais próximo para o mais distante.
– **Concluir tarefa —** o usuário informa o identificador da tarefa e o sistema altera o status de “Pendente” para “Concluída”, preservando o registro em vez de apagá-lo.
– **Remover tarefa —** permite excluir um registro criado por engano, sempre com confirmação antes da exclusão definitiva.
– **Resumo geral —** apresenta os totais de tarefas cadastradas, pendentes e concluídas, dando ao usuário a visão do conjunto que hoje ele não tem.

As funcionalidades foram distribuídas entre as três fases do desafio para que cada entrega tenha um escopo realista e verificável:

Quadro 1 — Funcionalidades do TaskTracker por fase do projeto

| Código | Funcionalidade | Descrição | Fase |
|---|---|---|---|
| F01 | Cadastrar tarefa | Registrar título, descrição, prioridade e data limite, com validação dos dados digitados. | 2 |
| F02 | Listar tarefas | Exibir todas as tarefas ordenadas por prioridade e por data limite. | 2 |
| F03 | Concluir tarefa | Alterar o status de uma tarefa de “Pendente” para “Concluída”. | 2 |
| F04 | Remover tarefa | Excluir uma tarefa após confirmação do usuário. | 2 |
| F05 | Resumo geral | Mostrar os totais de tarefas cadastradas, pendentes e concluídas. | 2 |
| F06 | Persistência em arquivo | Salvar e recarregar as tarefas para que não sejam perdidas ao fechar o programa. | 3 |
| F07 | Execução em contêiner | Empacotar a aplicação em uma imagem Docker para execução padronizada. | 3 |

Fonte: elaborado pelo autor (2026).

**Escopo desta entrega:** a Fase 1 é exclusivamente de planejamento. Nenhuma linha de código é escrita agora — o objetivo é definir com precisão o problema, os dados, as regras e a lógica, para que a implementação da Fase 2 seja apenas a tradução deste documento para a linguagem Python.



## 2 DECOMPOSIÇÃO DO PROBLEMA E DADOS DO SISTEMA

Esta seção decompõe o TaskTracker em seus três elementos fundamentais: o que entra no sistema (dados digitados pelo usuário), o que sai do sistema (informações devolvidas na tela) e quais regras o sistema deve respeitar para que os dados permaneçam corretos e coerentes.

### 2.1 Entradas de dados

São todas as informações que o usuário precisará digitar no sistema. Para cada entrada estão definidos o tipo, a obrigatoriedade, um exemplo e a validação que o sistema aplicará antes de aceitar o valor.

Quadro 2 — Entradas de dados do sistema

| Entrada de dados | Tipo | Obrigatóri o | Exemplo | Validação prevista |
|---|---|---|---|---|
| Opção do menu | Número | Sim | 1 | Deve ser um número de 1 a 6. |

| | | | | |
|---|---|---|---|---|
| | inteiro | | | |
| Título da tarefa | Texto | Sim | Estudar comandos de Git | Não pode ser vazio nem conter apenas espaços; até 60 caracteres. |
| Descrição da tarefa | Texto | Não | Revisar add, commit e push | Campo livre; pode ficar em branco. |
| Prioridade | Texto | Sim | Alta | Aceita somente Alta, Média ou Baixa. |
| Data limite | Data (DD/MM/AAAA) | Sim | 20/09/2026 | Deve ser uma data válida do calendário e não pode ser anterior à data atual. |
| Categoria | Texto | Não | Estudos | Campo livre; se ficar em branco, assume “Geral”. |
| Identificador (ID) da tarefa | Número inteiro | Sim | 3 | Deve corresponder a uma tarefa existente na lista. |
| Confirmação de exclusão | Texto (S/N) | Sim | S | Aceita apenas S (sim) ou N (não), em maiúscula ou minúscula. |

Fonte: elaborado pelo autor (2026).

## 2.2 Saídas de dados

São todas as informações que o sistema deverá mostrar na tela para o usuário — incluindo as mensagens de erro, que fazem parte da solução e não da falha.

Quadro 3 — Saídas de dados do sistema

| Saída de dados | Quando ocorre | Exemplo do que aparece na tela |
|---|---|---|
| Menu principal | Ao iniciar o programa e após cada operação concluída. | 1 - Cadastrar tarefa \| 2 - Visualizar tarefas \| 3 - Concluir tarefa \| 4 - Remover tarefa \| 5 - Resumo \| 6 - Sair |



| Saída de dados | Quando ocorre | Exemplo do que aparece na tela |
|---|---|---|
| Mensagem de confirmação de cadastro | Depois de gravar uma nova tarefa. | Tarefa cadastrada com sucesso! (ID: 4) |
| Lista de tarefas organizada | Quando o usuário pede a visualização e existe ao menos uma tarefa. | Tabela com as colunas ID \| Título \| Prioridade \| Data limite \| Status, ordenada por prioridade. |
| Aviso de lista vazia | Quando o usuário pede a lista e não há nenhuma tarefa | Nenhuma tarefa cadastrada até o momento. |

| | | |
|---|---|---|
| | cadastrada. | |
| Total de tarefas / resumo | Ao final da listagem e na opção de resumo. | Total: 7 tarefas \| Pendentes: 5 \| Concluídas: 2 |
| Mensagem de conclusão | Depois de marcar uma tarefa como concluída. | Tarefa 3 concluída com sucesso! |
| Mensagem de remoção | Depois de excluir uma tarefa confirmada pelo usuário. | Tarefa 5 removida do sistema. |
| Mensagens de erro de validação | Sempre que um dado digitado não atende às regras do sistema. | O título não pode ficar vazio. / Prioridade inválida: digite Alta, Média ou Baixa. / Não existe tarefa com o ID informado. |
| Mensagem de encerramento | Quando o usuário escolhe a opção Sair. | Obrigado por usar o TaskTracker. Até logo! |

Fonte: elaborado pelo autor (2026).

### 2.3 Regras do sistema

As regras abaixo definem o comportamento obrigatório do TaskTracker. Elas serão traduzidas diretamente em validações no código da Fase 2. **RN01. Título obrigatório.** O sistema não pode aceitar tarefas sem título. Se o usuário deixar o campo em branco ou digitar apenas espaços, o sistema deve exibir um aviso e solicitar o título novamente.

**RN02. Prioridade restrita a três valores.** A prioridade deve ser apenas Alta, Média ou Baixa. Qualquer outro valor é rejeitado e o sistema pede a informação de novo.

**RN03. Data limite válida.** A data deve seguir o formato DD/MM/AAAA, existir no calendário e não pode ser anterior à data atual.

**RN04. Identificador único.** Cada tarefa recebe um número de identificação (ID) único e sequencial, gerado pelo próprio sistema. Um ID nunca é reaproveitado, mesmo que a tarefa correspondente seja removida.

**RN05. Status inicial padronizado.** Toda tarefa recém-cadastrada nasce com o status “Pendente”; o usuário não digita o status.

**RN06. Conclusão não repetida.** Uma tarefa já concluída não pode ser concluída de novo; nesse caso o sistema informa que ela já está finalizada.



**RN07. Operação sobre ID inexistente.** Se o usuário informar um ID que não existe, o sistema exibe uma mensagem de erro e retorna ao menu, sem

encerrar o programa.

**RN08. Ordenação da listagem.** A lista é sempre exibida da prioridade mais alta para a mais baixa e, entre tarefas de mesma prioridade, da data limite mais próxima para a mais distante.

**RN09. Aviso de lista vazia.** Se não houver tarefas cadastradas, o sistema avisa o usuário em vez de exibir uma tabela vazia.

**RN10. Exclusão com confirmação.** A remoção de uma tarefa só é executada após confirmação explícita do usuário (S/N); se a resposta for N, a operação é cancelada.

**RN11. Opção de menu inválida.** Se o usuário digitar uma opção que não existe, o sistema mostra um aviso e exibe o menu novamente, sem travar nem encerrar.

**RN12. Execução contínua.** O sistema só encerra quando o usuário escolhe explicitamente a opção Sair.

**Observação sobre armazenamento:** no escopo previsto para a Fase 2 as tarefas ficam guardadas na memória do programa enquanto ele estiver em execução. A gravação em arquivo, para que os dados sobrevivam ao fechamento do programa, está planejada como funcionalidade F06 na Fase 3.



## 3 RECEITA LÓGICA DO SISTEMA (ALGORITMO EM PORTUGUÊS)

Esta seção descreve o passo a passo do funcionamento do TaskTracker em linguagem natural, como se fosse uma receita explicada para uma pessoa que nunca programou. Cada passo é uma ação única, na ordem exata em que deve acontecer — é essa sequência que será transformada em código Python na Fase 2.

### 3.1 Fluxo principal (menu do sistema)

**Passo 1.** Iniciar o programa.

**Passo 2.** Criar uma lista de tarefas vazia e um contador de identificadores começando em 1.

**Passo 3.** Exibir a mensagem de boas-vindas do TaskTracker.

**Passo 4.** Exibir o menu com as seis opções: cadastrar, visualizar, concluir, remover, resumo e sair.

**Passo 5.** Pedir ao usuário que digite o número da opção desejada. **Passo 6.** Verificar se a opção digitada é um número de 1 a 6; se não for, avisar que a opção é inválida e voltar ao Passo 4.

**Passo 7.** Se a opção for 1, executar a receita de cadastro (seção 3.2). **Passo 8.** Se a opção for 2, executar a receita de visualização (seção 3.3). **Passo 9.** Se a opção for 3, executar a receita de conclusão (seção 3.4). **Passo 10.** Se a opção for 4, executar a receita de remoção (seção 3.5). **Passo 11.** Se a

opção for 5, executar a receita de resumo (seção 3.6). **Passo 12.** Se a opção for 6, exibir a mensagem de despedida e encerrar o programa.

**Passo 13.** Enquanto o usuário não escolher a opção 6, voltar ao Passo 4 e repetir todo o ciclo.

### 3.2 Passo a passo do cadastro de tarefa

**Passo 1.** Pedir ao usuário o título da tarefa.

**Passo 2.** Verificar se o título não está vazio; se estiver, avisar que o título é obrigatório e pedir novamente.

**Passo 3.** Pedir a descrição da tarefa, informando que o campo é opcional e pode ficar em branco.

**Passo 4.** Pedir a prioridade da tarefa, indicando as três opções aceitas: Alta, Média ou Baixa.

**Passo 5.** Verificar se a prioridade digitada é uma das três opções; se não for, avisar e pedir novamente.



**Passo 6.** Pedir a data limite no formato DD/MM/AAAA.

**Passo 7.** Verificar se a data é válida e se não está no passado; se houver problema, avisar e pedir novamente.

**Passo 8.** Pedir a categoria da tarefa; se o usuário deixar em branco, usar “Geral”.

**Passo 9.** Gerar o próximo identificador (ID) disponível e somar 1 ao contador de identificadores.

**Passo 10.** Definir o status da nova tarefa como “Pendente”.

**Passo 11.** Guardar a tarefa na lista de tarefas, com todos os dados coletados.

**Passo 12.** Exibir a mensagem “Tarefa cadastrada com sucesso!”, informando o ID gerado.

**Passo 13.** Voltar ao menu principal.

### 3.3 Passo a passo da visualização das tarefas

**Passo 1.** Acessar a lista de tarefas.

**Passo 2.** Verificar se existe alguma tarefa cadastrada.

**Passo 3.** Se a lista estiver vazia, avisar o usuário com a mensagem “Nenhuma tarefa cadastrada até o momento” e voltar ao menu.

**Passo 4.** Se houver tarefas, ordenar a lista pela prioridade (Alta, depois Média, depois Baixa) e, em caso de empate, pela data limite mais próxima.

**Passo 5.** Exibir o cabeçalho da tabela com as colunas ID, Título, Prioridade, Data limite e Status.

**Passo 6.** Mostrar as tarefas uma por uma na tela, cada uma em sua própria linha.

**Passo 7.** Exibir, ao final da lista, o total de tarefas, quantas estão pendentes e quantas foram concluídas.

**Passo 8.** Voltar ao menu principal.

### 3.4 Passo a passo da conclusão de uma tarefa

**Passo 1.** Exibir a lista de tarefas, para que o usuário veja os identificadores disponíveis.

**Passo 2.** Verificar se existe pelo menos uma tarefa pendente; se não existir, avisar e voltar ao menu.

**Passo 3.** Pedir ao usuário o ID da tarefa que deseja concluir.

**Passo 4.** Procurar na lista a tarefa que possui esse ID.



**Passo 5.** Se nenhuma tarefa for encontrada, avisar que o ID não existe e voltar ao menu.

**Passo 6.** Se a tarefa encontrada já estiver com o status “Concluída”, avisar que ela já foi finalizada e voltar ao menu.

**Passo 7.** Alterar o status da tarefa de “Pendente” para “Concluída” e registrar a data da conclusão.

**Passo 8.** Exibir a mensagem de sucesso informando o ID e o título da tarefa concluída.

**Passo 9.** Voltar ao menu principal.

### 3.5 Passo a passo da remoção de uma tarefa

**Passo 1.** Exibir a lista de tarefas com os respectivos identificadores. **Passo 2.** Pedir ao usuário o ID da tarefa que deseja remover.

**Passo 3.** Procurar a tarefa correspondente na lista.

**Passo 4.** Se a tarefa não existir, avisar o usuário e voltar ao menu. **Passo 5.** Mostrar o título da tarefa encontrada e perguntar se o usuário confirma a exclusão (S/N).

**Passo 6.** Se a resposta for N, cancelar a operação, avisar que nada foi removido e voltar ao menu.

**Passo 7.** Se a resposta for S, retirar a tarefa da lista.

**Passo 8.** Exibir a mensagem confirmando a remoção.

**Passo 9.** Voltar ao menu principal.

### 3.6 Passo a passo do resumo geral

**Passo 1.** Acessar a lista de tarefas.
**Passo 2.** Contar quantas tarefas existem no total.
**Passo 3.** Contar quantas tarefas estão com o status “Pendente”.
**Passo 4.** Contar quantas tarefas estão com o status “Concluída”. **Passo 5.** Contar quantas tarefas pendentes têm prioridade Alta.
**Passo 6.** Exibir os totais na tela, em linhas separadas e com rótulos claros. **Passo 7.** Voltar ao menu principal.

### 3.7 Representação em pseudocódigo

Para tornar a passagem do português para a linguagem Python ainda mais direta, o fluxo principal e as rotinas de cadastro, visualização e conclusão estão representados abaixo em pseudocódigo:

Quadro 4 — Pseudocódigo do fluxo principal do TaskTracker



```
INICIO
    lista_tarefas <- lista vazia
    proximo_id <- 1
    ESCREVA "Bem-vindo ao TaskTracker"

    REPITA
        ESCREVA "1-Cadastrar 2-Visualizar 3-Concluir 4-Remover 5-Resumo 6-Sair"
        LEIA opcao

        SE opcao = 1 ENTAO
            LEIA titulo
            ENQUANTO titulo = "" FACA
                ESCREVA "O titulo nao pode ficar vazio."
                LEIA titulo
            FIM ENQUANTO

            LEIA descricao

            LEIA prioridade
            ENQUANTO prioridade NAO ESTA EM ("Alta", "Media", "Baixa") FACA
                ESCREVA "Prioridade invalida: digite Alta, Media ou Baixa."
                LEIA prioridade
            FIM ENQUANTO

            LEIA data_limite
               ENQUANTO data_limite NAO E VALIDA OU data_limite < data_de_hoje
              FACA ESCREVA "Data invalida ou anterior a hoje. Use DD/MM/AAAA."
                LEIA data_limite
            FIM ENQUANTO

            tarefa <- (proximo_id, titulo, descricao, prioridade, data_limite,
```

```
 "Pendente" )
            ADICIONE tarefa EM lista_tarefas
            ESCREVA "Tarefa cadastrada com sucesso! ID:", proximo_id
            proximo_id <- proximo_id + 1

        SENAO SE opcao = 2 ENTAO
            SE lista_tarefas ESTA VAZIA ENTAO
                ESCREVA "Nenhuma tarefa cadastrada ate o momento."
            SENAO
                ORDENE lista_tarefas POR prioridade, DEPOIS POR data_limite
                PARA CADA tarefa EM lista_tarefas FACA
                    ESCREVA tarefa
                FIM PARA
                ESCREVA "Total:", CONTE(lista_tarefas)
            FIM SE

        SENAO SE opcao = 3 ENTAO
            LEIA id_escolhido
            SE EXISTE tarefa COM id = id_escolhido ENTAO
                SE tarefa.status = "Concluida" ENTAO
                    ESCREVA "Esta tarefa ja foi concluida."
                SENAO
                    tarefa.status <- "Concluida"
                    ESCREVA "Tarefa concluida com sucesso!"
                FIM SE
            SENAO
                ESCREVA "Nao existe tarefa com o ID informado."
            FIM SE

        SENAO SE opcao = 6 ENTAO
            ESCREVA "Obrigado por usar o TaskTracker. Ate logo!"
        SENAO
```



```
            ESCREVA "Opcao invalida. Escolha um numero de 1 a 6."
        FIM SE

    ATE opcao = 6
FIM
```

Fonte: elaborado pelo autor (2026).



## 4 MAPEAMENTO ARQUITETURAL PARA GIT E GITHUB

Esta seção define, antes de qualquer linha de código, como o projeto será organizado no repositório que será criado no GitHub na Fase 2. Definir a estrutura agora evita que o projeto cresça de forma desorganizada e garante que cada arquivo tenha um lugar previsto e uma finalidade clara.

### 4.1 Desenho da árvore do repositório

Estrutura de pastas e arquivos que será enviada ao GitHub na Fase 2:

Quadro 5 — Estrutura de pastas e arquivos do repositório

```
tasktracker-projeto/
|
|-- README.md # Apresentacao do projeto no GitHub |-- .gitignore # Arquivo
para ignorar arquivos temporarios |
|-- docs/
| `-- planejamento_logico.pdf # Este planejamento da Fase 1
|
`-- src/
    `-- main.py # Arquivo principal do programa Python
```

Fonte: elaborado pelo autor (2026).

Figura 1 — Estrutura do repositório tasktracker-projeto




Fonte: elaborada pelo autor (2026).

### 4.1.1 Finalidade de cada item da estrutura

Quadro 6 — Finalidade de cada item da estrutura do repositório



| Item | Tipo | Finalidade |
|---|---|---|
| tasktracker-projeto/ | Pasta raiz | Diretório principal do repositório; é a pasta que será versionada com Git e publicada no GitHub. |
| README.md | Arquivo | Apresentação do projeto no GitHub: o que é o TaskTracker, qual problema resolve, como executar o programa e qual o estágio atual do desenvolvimento. |
| .gitignore | Arquivo | Lista dos arquivos e pastas que o Git deve ignorar (arquivos temporários, cache do Python como __pycache__, configurações locais do editor), para que o repositório contenha apenas o que interessa ao projeto. |
| docs/ | Pasta | Concentra a documentação do projeto, separada do código-fonte. |

| | | |
|---|---|---|
| docs/planejamento_logico.p df | Arquivo | Este documento de planejamento lógico da Fase 1, versionado junto ao código para servir de referência durante a implementação. |
| src/ | Pasta | Concentra o código-fonte (source) da aplicação, isolado da documentação. |
| src/main.py | Arquivo | Arquivo principal do programa em Python: exibe o menu, lê as opções do usuário e executa as rotinas de cadastro, listagem, conclusão e remoção de tarefas. |

Fonte: elaborado pelo autor (2026).

### 4.1.2 Evolução prevista da estrutura (Fases 2 e 3)

À medida que o projeto avança, a mesma estrutura será ampliada de forma controlada. A evolução já prevista é:

Quadro 7 — Evolução prevista da estrutura nas Fases 2 e 3

```
tasktracker-projeto/
|-- README.md
|-- .gitignore
|-- requirements.txt # (Fase 2) bibliotecas necessarias ao projeto |--
Dockerfile # (Fase 3) receita da imagem do container |-- docs/
| |-- planejamento_logico.pdf
| `-- imagens/ # capturas de tela do sistema em execucao |-- src/
| |-- main.py # menu e fluxo principal
| `-- tarefas.py # (Fase 2) funcoes de cadastro, listagem e conclusao `-- tests/
          `-- test_tarefas.py # (Fase 2) testes iniciais das regras do sistema
```

Fonte: elaborado pelo autor (2026).

**Importante:** a estrutura da seção 4.1 é a que será efetivamente criada no início da Fase 2. Os itens marcados acima como Fase 2 e Fase 3 estão registrados apenas como planejamento de crescimento, para que a organização do repositório não precise ser refeita mais adiante.

## 4.2 Modelo conceitual do Git



Antes de usar os comandos, é necessário entender o modelo que está por trás deles. O Git não guarda apenas a versão mais recente de cada arquivo: ele organiza o projeto em quatro áreas distintas, e cada comando serve para mover o trabalho de uma área para a seguinte. Compreender essa divisão é o que diferencia usar o Git por repetição de usá-lo com consciência do que está acontecendo.

### 4.2.1 As quatro áreas do Git

Quadro 8 — As quatro áreas do Git

| Área | O que é | Como se chega até ela |
|---|---|---|
| 1. Diretório de trabalho | A pasta do projeto no computador, com os arquivos visíveis e editáveis. É o único lugar onde os arquivos são realmente escritos e alterados. | É onde o trabalho acontece; nenhum comando é necessário. |
| 2. Área de preparação (staging) | Uma lista intermediária que reúne exatamente as alterações escolhidas para entrar no próximo commit. Permite separar o que já está pronto do que ainda está inacabado. | git add |
| 3. Repositório local (.git) | O histórico completo do projeto, gravado no próprio computador. Cada commit é um ponto salvo e permanente, com autor, data e mensagem. | git commit |
| 4. Repositório remoto (GitHub) | A cópia do repositório hospedada no GitHub, que funciona ao mesmo tempo como backup, ponto de colaboração e vitrine profissional do projeto. | git push |

Fonte: elaborado pelo autor (2026).

### 4.2.2 O caminho de uma alteração até o GitHub



Figura 2 — As quatro áreas do Git e o caminho de uma alteração




Fonte: elaborada pelo autor (2026).

O caminho inverso, do GitHub de volta para o computador, é feito com git clone (quando o projeto ainda não existe na máquina) e com git pull (quando ele já existe e precisa ser atualizado).

### 4.2.3 Conceitos centrais do modelo

Quadro 9 — Conceitos centrais do modelo do Git

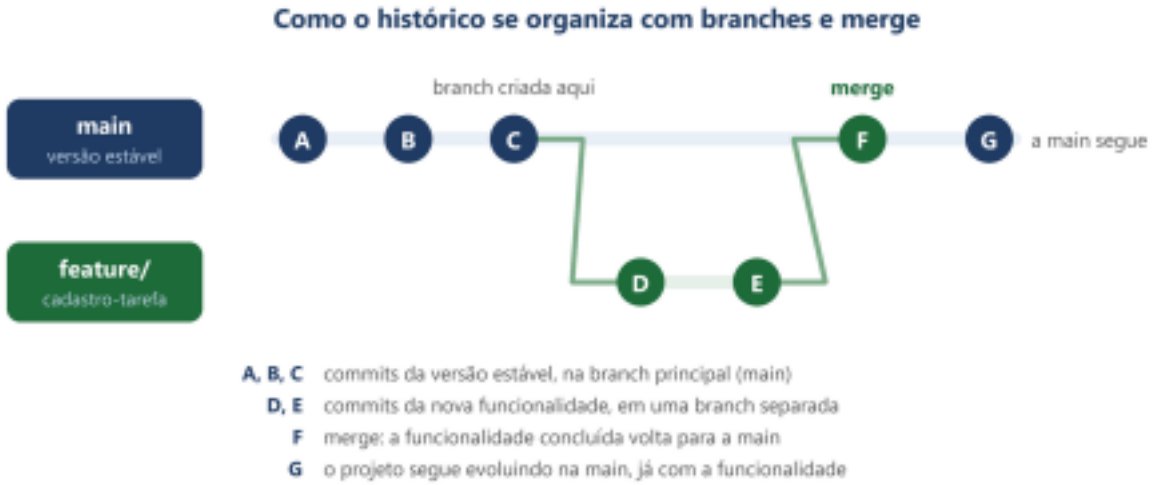
| Conceito | Significado |
|---|---|
| Commit | O registro de um estado do projeto em um determinado momento, com identificador próprio, autor, data e mensagem. Cada commit aponta para o commit anterior, e é essa ligação que forma o histórico como uma corrente contínua. |
| Mensagem de commit | A explicação do porquê da alteração. É o que transforma o histórico em algo legível meses depois, em vez de uma lista de mudanças sem sentido. |
| Branch | Uma linha de desenvolvimento paralela: um ponteiro para um commit, a partir do qual o trabalho segue em separado. Criar uma branch permite desenvolver uma funcionalidade nova sem colocar em risco a versão que já funciona. |
| main | A branch principal do repositório, onde fica a versão estável do projeto. |
| HEAD | O indicador da posição atual: aponta para a branch e o commit que estão refletidos no diretório de trabalho neste momento. |
| Merge | A junção de uma branch de volta à principal, quando a funcionalidade está concluída. |
| Conflito | A situação em que duas linhas de desenvolvimento alteraram as mesmas linhas do mesmo arquivo. O Git não decide sozinho: ele marca o trecho e o desenvolvedor escolhe qual versão permanece. |
| .gitignore | A lista do que deve ficar de fora do histórico (arquivos temporários, cache, configurações locais), para que o repositório contenha apenas o que pertence ao projeto. |



Fonte: elaborado pelo autor (2026).

### 4.2.4 Como o histórico se organiza com branches

Figura 3 — Organização do histórico com branches e merge




Fonte: elaborada pelo autor (2026).

### 4.2.5 Como este modelo será aplicado ao TaskTracker na Fase 2

Quadro 10 — Aplicação do modelo do Git ao TaskTracker na Fase 2

| Etapa | Comando previsto | O que acontece |
|---|---|---|
| 1. Criar o repositório local | git init | A pasta tasktracker-projeto passa a ser controlada pelo Git. |
| 2. Registrar o estado inicial | git add . → git commit | O README.md e o .gitignore entram no histórico como primeiro commit. |
| 3. Conectar ao GitHub | git remote add origin <url> | O repositório local passa a conhecer o repositório remoto. |
| 4. Publicar o projeto | git push -u origin main | Os commits sobem para o GitHub e o projeto fica visível. |
| 5. Desenvolver uma funcionalidade | git checkout -b feature/cadastro-tarefa | Cria uma linha paralela para implementar o cadastro sem afetar a main. |
| 6. Salvar o progresso | git add → git commit | Cada passo do algoritmo concluído vira um commit com mensagem descritiva. |
| 7. Integrar o que ficou pronto | git merge feature/cadastro-tarefa | A funcionalidade concluída volta para a branch principal. |
| 8. Consultar o histórico | git log | Mostra a sequência de commits que construiu o projeto. |

Fonte: elaborado pelo autor (2026).

**Ligação com a seção 3:** cada passo da receita lógica descrita na seção 3 se tornará um commit próprio na Fase 2. O algoritmo define o que implementar; este modelo define como esse trabalho será registrado, versionado e publicado.

## 4.3 Plano de aprendizado para Git e GitHub

Espero, na Fase 2, deixar de tratar o versionamento como um detalhe burocrático e passar a entendê-lo como parte do próprio trabalho de desenvolvimento. Concretamente, quero compreender o que é um commit — não apenas o comando que o cria, mas a ideia de registrar uma alteração pequena e coerente, com uma mensagem que explique por que ela foi feita, de modo que o histórico do projeto possa ser lido depois como a narrativa da sua construção. A partir disso, pretendo dominar o ciclo básico de salvar alterações de forma organizada com git add, git commit e git push, aprender a criar branches para desenvolver cada nova funcionalidade em separado sem colocar em risco a versão que já funciona, entender como integrar esse trabalho de volta e como resolver os conflitos de código que aparecem quando duas alterações tocam o mesmo arquivo. Quero também publicar o projeto no GitHub e usar o repositório remoto de verdade: como fonte única do código, como cópia de segurança e como vitrine profissional do que fui capaz de construir. Por fim, espero desenvolver a familiaridade necessária para navegar pelo histórico de versões e comparar estados anteriores do projeto — porque a segurança de poder voltar atrás sem perder nada é justamente o que permite experimentar e corrigir com tranquilidade, e essa confiança é a diferença prática entre programar sozinho em uma pasta solta e trabalhar como desenvolvedor profissional.

### 4.3.1 Objetivos específicos de aprendizado

Quadro 11 — Objetivos específicos de aprendizado de Git e GitHub

| Tópico | O que pretendo saber fazer ao final da Fase 2 |
|---|---|
| Repositório local e remoto | Inicializar um repositório com git init, conectá-lo ao GitHub e enviar o projeto com git push. |
| Commits | Registrar alterações pequenas e coerentes, com mensagens descritivas e úteis. |
| Branches | Criar uma branch por funcionalidade, trabalhar nela em isolamento e integrá-la à principal. |
| Conflitos de código | Identificar um conflito, entender o que cada lado alterou e resolvê-lo com segurança. |
| Histórico de versões | Consultar o histórico com git log, comparar versões e retornar a um estado anterior quando necessário. |
| Documentação no GitHub | Escrever um README.md que explique o projeto com clareza para quem chega de fora. |

Fonte: elaborado pelo autor (2026).

# REFERÊNCIAS

ASCENCIO, Ana Fernanda Gomes; CAMPOS, Edilene Aparecida Veneruchi de. **Fundamentos da programação de computadores**: algoritmos, Pascal, C/C++ e Java. 3. ed. São Paulo: Pearson Education do Brasil, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

CHACON, Scott; STRAUB, Ben. **Pro Git**. 2. ed. Berkeley: Apress, 2014. Disponível em: https://git-scm.com/book/pt-br/v2. Acesso em: 11 set. 2026.

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. **Lógica de programação**: a construção de algoritmos e estruturas de dados. 3. ed. São Paulo: Pearson Prentice Hall, 2005.

GITHUB. **GitHub Docs**: começar a usar o Git. Disponível em: https://docs.github.com/pt/get-started. Acesso em: 11 set. 2026.

PYTHON SOFTWARE FOUNDATION. **The Python tutorial**. Disponível em: https://docs.python.org/pt-br/3/tutorial/. Acesso em: 11 set. 2026.